

The Agentic Development Manifesto

Craftsmanship for the Age of AI
Agents



White Paper v1.0 — Singapore edition — June 2026



Table of contents

Executive summary	3
Chapter 1 — Productivity in the AI era	4
Chapter 2 — TCO and on-prem under Singapore's space constraints	5
Chapter 3 — Organising development teams	11
Chapter 4 — The AI-driven software development lifecycle	13
Chapter 5 — Specification Driven Development (SDD)	16
Chapter 6 — The AI IDP (Internal Developer Platform)	18
Chapter 7 — PDLC governance and metrics	20
Conclusion — An AI maturity checklist for CTOs	22
Appendix	23

Executive summary

Generative AI is accelerating software development — but without an adapted methodology, it can create as many problems as it solves. This document seeks to contribute a framework for turning that acceleration into durable organisational performance.

While Zenika's overarching **AI Multiplier framework** covers the full operational scope of an AI transformation across three complementary disciplines — **Shape** (strategy, discovery, design), **Ship** (engineering and delivery), and **Sync** (architectures and platforms), this document will focus on the **Ship** discipline and its **intersections** with the two other disciplines.

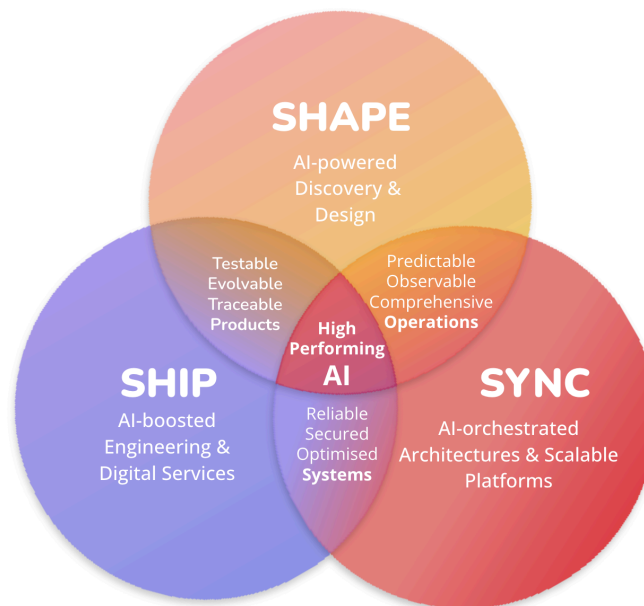


Figure 1: AI Multiplier framework: three disciplines whose intersection produces a multiplier effect.



Key takeaways

 **AI without methodology accelerates technical debt.** *Specification Driven Development* turns individual productivity into durable organisational performance.

 **Depend on no one.** A pluggable AI platform — gateway, registry, context factory — guarantees the freedom to switch model, provider, or cloud at any time.

 **Craftsmanship + AI = ❤️.** Curated shared instructions scale quality beyond individual discipline.

 **Measure value, not volume.** AI should be measured by the systemic friction it eliminates, not by the code it generates.

Chapter 1 — Productivity in the AI era

AI accelerates — but who verifies?

1.1. The productivity paradox

The market for generative AI applied to software development has reached a tipping point. Adoption figures are spectacular: 72% of developers use AI daily (Sonar 2026), 84% use or plan to use AI tools (Stack Overflow 2025), 88% of enterprises deploy AI in at least one function (McKinsey 2025), 21% more tasks are completed per developer (DORA 2025).

However, the figures below show a paradox surrounding these productivity gains. While AI-generated software is often deployed without rigorous review, most developers do not have a high trust level in what is generated.



1.2. The invisible debt

Teams that generate more code with AI experience *more* production failures, not fewer. Only 48% of developers systematically verify code before committing (Sonar 2026), and 66% are frustrated by “almost correct but not quite” solutions (Stack Overflow 2025) — code is written faster, but reviews, tests, and deployment do not keep pace. DORA conceptualises this phenomenon in 2026 with the phrase **verification tax**: the effort saved on boilerplate code is replaced by the cognitive load required to audit code that *looks* correct.

AI, trained on existing code that already contains defects, generates more technical debt per line than human developers. For organisations already burdened with technical debt, **20% to 40% of the IT budget** is absorbed by that debt.



Our convictions at Zenika

Faced with this accountability crisis, the answer lies neither in abandoning AI nor in blindly adopting more tools. It lies in **a standardised, vendor-independent, and verifiable methodology**: moving from opportunistic AI usage to a **methodical industrialisation of AI-assisted development**, anchored by a methodology that makes specifications the new pivot of every long-term development effort (*Specification Driven Development*), and by a pluggable technical framework.



Chapter 2 — TCO and on-prem under Singapore's space constraints

Keeping vendor choice, data residency, and cost trajectory under control in a city-state where every rack of metal carries a real-estate premium.

Agentic AI is expensive, and its ROI depends as much on cost control as on methodological effort. Particular to Singapore is the fact that **physical space here is scarce**. An analysis of the on-prem question in the local context is thus particularly pertinent.

2.1. The SaaS trap: a TCO analysis

The massive adoption of SaaS code assistants creates a structural dependency whose real cost few organisations measure. GitHub Copilot and Claude Code have established themselves as de facto standards, but their business model relies on a progressive lock-in mechanism. In 2026, the **silent subsidisation phase** of agentic models is ending across all major vendors in favour of **usage-based billing** indexed on actual token consumption, which penalises intensive use.

The total cost of ownership (TCO) can be seen as aggregating five dimensions over time:

1. the **licence cost** — now usage-based, therefore directly proportional to the team's agentic usage intensity
2. the **infrastructure cost** — compute, GPU, storage, maintenance operations, updates — minimal in SaaS, structural in on-prem, and particularly heavy in Singapore where colocation rates rank among the world's highest
3. the **migration cost** — the effort to exit a provider when prices or terms change — a proxy for *vendor lock-in*
4. the **IP protection** — effort budgeted to prevent any source code or documentation flowing through third-party servers from being exfiltrated, logged, or reused for training
5. the **methodology & tuning risk** — a cost present in every scenario (shared instructions, prompt calibration, deterministic verification) that grows substantially in the on-prem scenario to close the residual quality gap of Open Weight models and the lower operational maturity of self-hosting stacks

These costs can be compared across four main provider models: US-based SaaS, SaaS via Chinese APIs, Hybrid, and On-Prem.

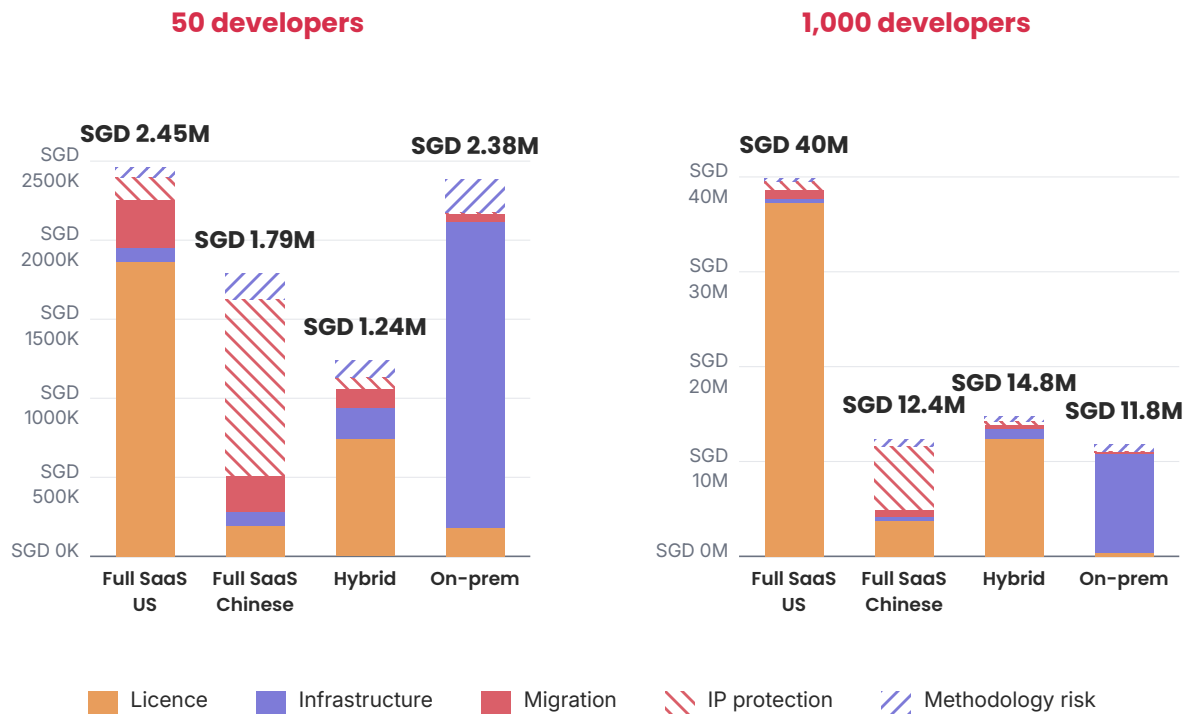


Figure 2: 3-year TCO by team size, after the usage-based pricing shift in 2026. Left: 50 developers (SMB / mid-market). Right: 1,000 developers (enterprise). Assumptions: weighted average $\approx$ SGD 900 / dev / month for US-headquartered frontier models (Anthropic, OpenAI, Google), $\approx$ SGD 90 for Chinese models via API; +15%/year consumption growth; fixed costs (methodology, infrastructure) amortised over team size. EUR-to-SGD conversion at 1.49. The On-prem scenario reflects Singapore colocation rates (CBRE **Global Data Center Trends 2025**), commercial electricity (EMA Singapore), and Green DC capacity constraints (IMDA **Green Data Centre Roadmap**, May 2024). Excludes uses other than code generation, and excludes CLOUD Act / cross-border mitigation overlays (cf. text).

Chinese models (Qwen3 Coder, GLM-4.6, DeepSeek, MiniMax) now produce results comparable to the best US frontier models on code generation. Raw token cost is roughly ten times lower, but IP and regulatory exposure explodes. For **Singapore organisations using Chinese model APIs hosted in mainland China**, the legal framework (Data Security Law, PIPL, National Intelligence Law) provides for government access without consent-based or PDPA-equivalent safeguards, and imposes outbound-data assessment obligations under the *Cyberspace Administration of China (CAC) Measures for the Security Assessment of Outbound Data Transfers*. For workloads handling sensitive data, raw token-cost arbitrage is overshadowed by compliance overhead and supervisory risk. **APAC-region deployments of Chinese providers (e.g., Alibaba Cloud Singapore)** materially reduce but do not fully eliminate this exposure, as cross-region traffic and inference telemetry may still transit mainland infrastructure. The real value option lies elsewhere: running those same open-weight models **self-hosted**, which folds back into the On-prem scenario.



The same exposure to US providers deserves equal attention. When data is hosted on AWS, Azure, or GCP — even in their Singapore regions — the provider remains subject to US legal process under the *Clarifying Lawful Overseas Use of Data Act* (CLOUD Act, 2018). A US-issued subpoena can compel disclosure of customer data regardless of where it physically resides. For Singapore enterprises processing sensitive code, trade secrets, or regulated data, this is a distinct risk category from commercial lock-in or pricing — and it cannot be mitigated by region choice alone. Mitigations exist (customer-managed keys via BYOK / External Key Stores, confidential computing, workload partitioning, routing the most sensitive flows to Singapore-jurisdiction providers) and add a quantifiable cost line to any TCO analysis.

The option of self-hosting Chinese models does demand a specific watchpoint. China's industrial strategy in early 2026 explicitly aims to decouple from the Nvidia ecosystem: the CAC has prohibited major local players from acquiring Nvidia hardware, while the United States likewise banned export of those cards into the Chinese market. As a consequence, Nvidia portability of Chinese open-weight models is likely to drift over the next 12 to 24 months (kernels, operators, and quantisation recipes finely tuned for Huawei's Ascend). To avoid being squeezed between CUDA's monopoly in the West and the rise of CANN in China, it will be crucial to rely on the work of players such as Red Hat, whose stated goal is precisely to develop agnostic inference frameworks capable of abstracting over both proprietary environments.

For Singapore operators specifically, this dual decoupling comes with concrete operational realities. Singapore is classified as Tier 2 under the (now-suspended) US AI Diffusion Rule, and BIS export controls on H100, H200, B100 and B200 remain in force through licensing and end-user certifications (NAC license exception, *End-User VEU* attestations). Procurement of frontier GPUs therefore requires planning around export-licence posture, with supplier lead times stretching from weeks to quarters depending on compliance review. Symmetrical Chinese-side export controls on Ascend, advanced lithography, and rare-earth supply make a single-source bet on Ascend equally fragile. A pragmatic hardware diversification posture — qualifying Nvidia, AMD MI, and (where authorised) Ascend in parallel — is therefore not a long-term strategic option but a near-term procurement-resilience question.

This dependency on foreign vendors could be justified if it translated into commensurate returns on investment. Yet McKinsey observes that only 21% of organisations have truly redesigned their workflows to integrate AI. The remaining 79% layer AI on top of existing processes, maximising dependency risk without reaping the organisational benefits.



2.2. The Singapore on-prem reality

Dedicated on-prem self-hosting in Singapore is in practice reserved for workloads whose regulatory posture justifies the premium such as critical national infrastructure. For most other enterprise AI workloads, “on-prem” in the operational sense is delivered by a **cloud Singapore-region deployment with customer-managed keys (BYOK/CMEK) and confidential computing**, or by **Singapore-jurisdiction providers** (Singtel/NCS, ST Engineering, StarHub-affiliated) — keeping data local without the real-estate exposure of physical racks.

Indeed, on-prem self-hosting in Singapore is structurally expensive, due to two main drivers:

Colocation real estate. Singapore consistently ranks among the world’s most expensive data-centre markets. CBRE’s *Global Data Center Trends 2025* reports asking rents of USD 310–470 per kW per month for Singapore, against USD ~168 for Frankfurt and USD ~196 for Northern Virginia. For a typical 30 kW cluster, the purely geographic premium (colocation + electricity + cooling) runs at **+70% vs Frankfurt and +120% vs Northern Virginia**.

Electrical capacity. A de facto moratorium 2019–2022, the IMDA *Green Data Centre Roadmap* (May 2024, at least 300MW of additional capacity with further growth contingent on green energy deployment), and successive *Data Centre Calls for Application* (DC-CFA 2023, DC-CFA2 2026) signal that capacity is allocated, not abundant. Vacancy hovers near 1% (Cushman & Wakefield / CBRE, 2024), and additions through 2030 are dominated by retrofit and mezzanine expansions, not greenfield. Securing rack space means planning around allocation cycles.

2.3. The Open Weight option: an economic lever

Open Weight models are now competitive enough to make the local-execution option an economic lever in its own right. Whether to use it, and for which workloads, is not primarily a matter of philosophy, but TCO and sensitivity.

Performance has caught up for enterprise code generation: Mistral Small 4 outperforms 120B models on LiveCodeBench while producing 20% fewer tokens, and DeepSeek Coder V2, Qwen 2.5/3.5, Llama 3.3, and SEA-LION (the AI Singapore + IMDA model specialised in South-East Asian languages) are solid alternatives. These models can run locally with 30 to 60 GB of RAM for the smaller variants, limiting third-party API dependency and the associated recurring costs.

A limit to keep in mind

The most powerful Open Weight models do not yet run on standard workstations: they require **dedicated infrastructure** (GPU, memory, network). The ground is moving fast — **continuous monitoring** is critical to adjust the local/cloud switchover thresholds and adapt best practices to ongoing hardware and software evolutions.

Two nuances must be acknowledged. First, at a reasonable execution cost (a local workstation, a departmental GPU server), **the best proprietary models keep a slight lead** on the most demanding tasks — the on-prem scenario must therefore budget additional effort for tuning, specialised embeddings, prompt calibration, and stronger methodology. Second, **the on-prem stack is new**: model self-hosting, AI Gateway, vector stores, and internal MLOps do not yet have the operational hindsight of established SaaS solutions — required skills are scarce on the Singapore market, and the risk of hardware or vendor obsolescence (acquisition, strategic pivot of an Open Weight provider) are real.



Singapore AI governance — key milestones

- **2018:** MAS *FEAT principles* for finance — origin of Singapore's sector-driven approach.
- **2022:** *AI Verify* toolkit launched — world-first government-developed AI testing framework.
- **Jan. 2026:** *Model AI Governance Framework for Agentic AI* launched at WEF — world-first.
- **Feb. 2026:** *National AI Council* established on four priority sectors.

Singapore's regime is *voluntary, sector-driven, and pro-innovation*. Enterprise compliance combines PDPA for personal data, sector-regulator guidance (MAS, MOH), IMDA frameworks as a governance signal, and parallel monitoring of US export controls (BIS) and Chinese cross-border data rules (CAC) — all of which translate into TCO line items rather than ideological choices.

Beyond the enterprise level, AI Singapore — the national AI programme run under NRF with IMDA — embeds the same posture in its national strategy: keeping the capacity to run frontier models on local infrastructure (including through SEA-LION for South-East Asian language coverage) is treated as a hedge against vendor and geopolitical concentration.

Nationally as in enterprises, preserving the option to run models locally protects future freedom of choice. Individually too, 57% of developers worry about sensitive data leaking through SaaS AI tools (Sonar 2026). Local processing of source code directly addresses that concern and simplifies **PDPA compliance, MAS data-residency obligations under the Outsourcing and TRM Guidelines, and CLOUD Act subpoena exposure**.

2.4. The Zenika pluggable approach

We strongly believe that **the goal is not to reject major vendors' solutions, but never to depend solely on them**.

The approach we recommend is “pluggable best-of-breed.” It consists of three principles:



Abstraction

An **AI Internal Developer Platform (AI IDP)** layer decouples the development workflow from the model provider. Switching from a proprietary model to an Open Weight model, from a cloud API to local execution, happens via configuration.



Agnostic tooling

The framework relies on open-source and interoperable tools rather than locked proprietary IDEs. Every brick is replaceable.



Permanent Plan B

An organisation should, at any moment, be able to migrate its workflow to local models without loss of functionality. This capability should be tested and maintained.

Our field experience confirms this stance. Zenika supports many organisations that already master their own infrastructure for data-residency or regulatory reasons (healthcare, finance, public sector...). In most cases, the **hybrid approach** paired with a **clear and controlled data governance**

policy makes the most sense. Risk is mitigated by clear data flow control and maintaining the ability to switch providers.

In a world where the hardware supply chain remains global, AI is in the end not too different from other resources in Singapore — pure self-reliance is neither the goal nor achievable. What is achievable is **gradual, switchable vendor independence**: a posture that protects against diverse sources of operational risk such as unilateral price hikes, changes to terms of service, US CLOUD Act subpoenas reaching data hosted in Singapore regions, Chinese cross-border data-transfer requirements when using mainland model APIs, BIS export controls restricting access to frontier GPU hardware, and future regulatory requirements.



When does on-prem actually pay off?

On-prem self-hosting in Singapore earns its real-estate premium under three combined conditions, not before:

1. **Regulatory pull**: workloads where local retention and direct operational control are non-negotiable and cloud contractual controls are deemed insufficient by the regulator or the institution's own risk posture.
2. **Scale**: roughly 200–300 developers or more, the inflection at which colocation and MLOps overhead amortise across enough usage to undercut Hybrid (cf. figure 2, enterprise panel).
3. **Operating maturity**: an in-house MLOps team or a qualified partner already positioned in Singapore, with secured Green DC allocation.

Outside these conditions, Hybrid with routing across cloud Singapore region (BYOK/CMEK), local providers (Singtel/NCS, ST Engineering), and selective open-weight self-hosting is the pragmatic default.

Chapter 3 — Organising development teams

Where does your organisation stand, and what boundaries is AI redrawing?

To tame agentic AI and turn it into ROI leverage, you first need to know where you start from. The matrix that follows positions any organisation on that path.

3.1. AI maturity matrix: where does your organisation stand?

To help decision-makers position themselves, we offer a four-level self-diagnostic grid — *Explorer*, *Structurer*, *Facilitator*, *Propeller* — presented below from the most advanced (top) to the most basic (bottom), with a five-axis radar to visualise one's profile. The aim is to situate the organisation in its AI-assisted development practices, from individual use to the deployment of enterprise tools, methodologies, and an internal developer platform favouring generative AI (AI IDP), in order to chart a trajectory aimed at optimising those practices.

MATURITY LEVELS

4

Propeller

AI-assisted development is fully integrated into the engineering culture. Multi-agent chains (orchestrator + specialised sub-agents) run in production, and the AI IDP (gateway, registry, context factory) secures vendor independence. The team is an AI reference in its ecosystem.

3

Facilitator

A structured workflow frames production: the specification precedes generation, validation is systematic (tests, static analysis, review), and conventions are versioned through curated shared instructions. Vendor independence is identified as a strategic concern.

2

Structurer

The organisation has deployed one or more code assistants (Copilot, Cursor) on targeted tasks. Verification exists but remains partial; individual productivity gains mask growing technical debt.

1

Explorer

AI is used individually by a few curious developers, without framework or governance. Tools are chosen on personal preference, and knowledge does not propagate to the team.



Figure 3: Maturity radar and description of the four levels. The illustrated profile (red dashed line) corresponds to a typical organisation at the Structurer stage.



Gartner forecast

40% of enterprise applications will integrate specific AI agents by the end of 2026, vs. less than 5% in 2025.

But the same study warns that **40%** of agentic AI projects will be abandoned by the end of 2027 — precisely because they lack a methodological framework.

The gap between **Level 2 — Structurer** (where most organisations sit, having deployed Copilot, Cursor, or equivalents without a methodological framework) and **Level 4 — Propeller** (where the framework and the AI IDP are operational) is precisely the risk zone Gartner is pointing to. Without methodology and vendor independence, pilots never cross the scale-up threshold.

3.2. The fusion of roles around the specification

Conway's law states that software systems reflect the communication structure of the organisation that produces them. In the AI-assisted development era, this law takes on a new dimension: if the methodology changes, the organisation must follow.

The specification artifact becomes the convergence point of these three roles. Reviews, validations, and iterations organise themselves around it — replacing traditional silos with a continuous flow.

McKinsey reveals that 51% of enterprises have already observed negative consequences from AI adoption, and only 33% successfully deploy it at organisational scale. The remaining two thirds stay stuck in "permanent pilot." The cause is not technical — it is organisational. Until roles reconfigure themselves around the specification, scaling remains out of reach.

Specification Driven Development redraws the boundaries between traditional roles.



The Developer

Is no longer a code writer. They become a *specification editor* who defines intentions, validates generated artifacts, and maintains system consistency.



The Architect

No longer just drafts diagrams. They become a *context engineer* whose value lies in their ability to turn knowledge into actionable context for agents.



The Product Owner

Gains autonomy: responsible for business intent, expressed in a clear specification language that reduces friction with the technical team.

How these roles interact in the development lifecycle will be further explored in the next section.

Chapter 4 — The AI-driven software development lifecycle

The AI Product Development LifeCycle (PDLC) redefines the processes and roles of software production.

Taming agentic AI starts with mapping what it displaces: not one more tool in the IDE, but a recomposition of **processes and roles** along the production cycle.

Introducing AI into the software development lifecycle is not a mere tooling addition. It is a recomposition of the production chain: context sources diversify, intermediate artifacts multiply, and deterministic control points become indispensable.

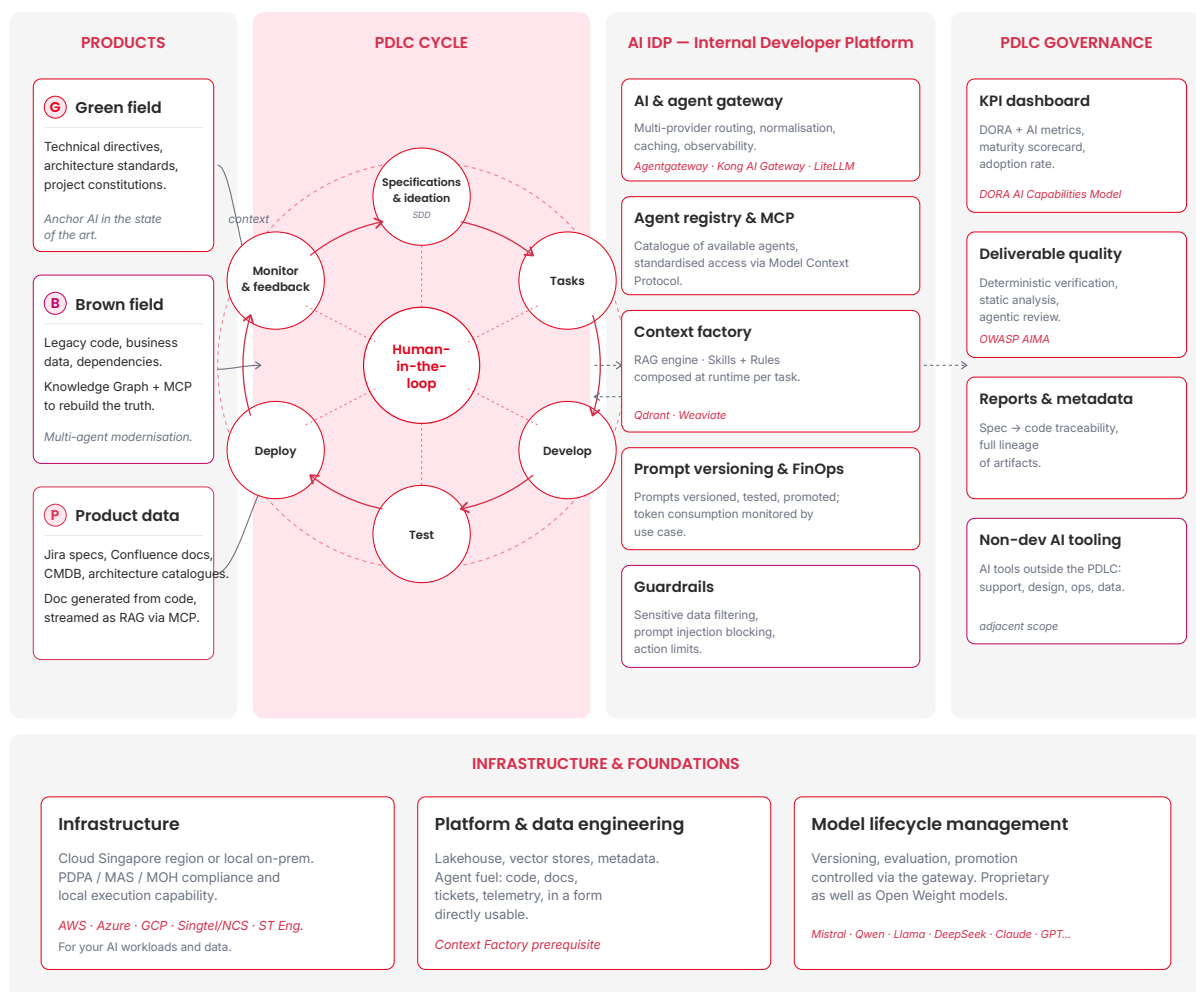


Figure 4: Functional diagram of the AI PDLC: products (Green field, Brown field), development cycle with human-in-the-loop, AI IDP platform (gateway, registry, context factory, prompt versioning, guardrails), governance, and infrastructure foundations.

The diagram articulates four zones: **context sources** (green/brown field) feed the **PDLC cycle** orchestrated by agents with a **human-in-the-loop** at every stage; the **AI IDP** provides the orchestration bricks (Chapter 6); **governance** measures and controls (Chapter 7); the **infrastructure** delivers execution autonomy, covered below.



4.1. Infrastructure and foundations

The AI PDLC can only deliver on its portability and data-residency promises if it rests on infrastructure compatible with Singaporean regulatory requirements, aware of US export and CLOUD Act exposures, and configurable for Chinese cross-border data-transfer obligations when APAC routing is in play. Three families of bricks make up this foundation.



Platform & data engineering

Lakehouse, vector stores, metadata: the fuel for agents (code, documents, telemetry, tickets) must be exposed in a directly usable form.



Model lifecycle

Proprietary as well as open-weight models (Mistral, DeepSeek, Qwen, Llama, SEA-LION) are versioned, evaluated, and promoted like application code, deployed under control via the *gateway*.



Cloud & on-prem hosting

AWS, Azure, and GCP Singapore regions are the default infrastructure for AI workloads, with **Government Commercial Cloud (GCC)** wrapping them in a regulated perimeter for public-sector use. For workloads sensitive to **US CLOUD Act exposure** or requiring strict local-only handling, **Singtel (NCS)**, **ST Engineering**, and **StarHub-affiliated** providers offer Singapore-headquartered hosting under PDPA jurisdiction. **Alibaba Cloud Singapore** region is a legitimate option for APAC-centric workloads, with the caveat that any flow back to mainland China regions triggers PIPL and CAC outbound-transfer obligations.

4.2. Framework use cases: Green Field vs Brown Field

The framework structures the approach according to the context the AI must ingest. For a **green field** project, the context does not yet exist: the context engineer (the architect) anchors the AI in the state of the art by codifying technical directives, architecture standards (ADRs, security, observability), and a project constitution — the whole versioned with the code as **curated shared instructions**.

For a **brown field** project — most enterprise cases — the challenge is no longer to define a direction, but to **rebuild the business truth** from an existing system.

Beyond code, **product data** also feeds the cycle: specifications are typically scattered (Jira, Confluence, CMDB, architecture catalogues) and documentation goes stale. Three data workstreams accompany adoption — clean up the existing patrimony and appoint the missing Data Owners/Stewards; generate documentation from the code and version it with it; expose those sources to agents by streaming them to a RAG capability accessible via MCP.



The brown field challenge

Generative AI is trained on recent public code; it knows nothing of the enterprise's legacy system. Without structured context, it hallucinates. With poorly framed context, it propagates debt.

Two mechanisms address this challenge: **Knowledge Graphs** extract dependencies, schemas, and business rules so the AI navigates the real architecture rather than inferring from a handful of files; **deterministic analysis via MCP** gives agents access to static analysis, search, or test execution tools with verifiable results. On this base, **multi-agent architectures** take charge of modernisation — extraction of business rules, target specification, generation, non-regression verification — each within a bounded context, under human control.

4.3. The Zenika edge

Facing a fragmented market split between proprietary tools and partial approaches, Zenika offers an integrated approach that combines five coherent pillars: the methodology produces verifiable artifacts, the framework makes them portable, the industrialised practices ensure their diffusion at scale, an enriched CI/CD installs deterministic verification loops (AI quality gates, spec ↔ artifact traceability) at the heart of the delivery pipeline, and a complete product loop carries usage signals back into the specification.



Structured methodology

SDD redefines the development cycle around the specification as a central, verifiable, and auditable artifact — not a mere prompt optimiser.



Modular framework

The AI IDP — gateway, registry, context factory — guarantees the freedom to switch at any time, without redesigning delivery.



Industrialised practices

Standards diffused through **curated shared instructions** versioned in the code, and team-level **Skills**.



Enriched CI/CD

Pipeline augmented with **asynchronous processes** (agentic review, spec ↔ code alignment) that maintain consistency over time.



Complete product loop

From feedback collection (telemetry, verbatim) to augmented deployment (feature flags, canary release, piloted rollback), agents feed the specifications — which sets enterprise AI practices apart from a mere developer toolkit.



Chapter 5 — Specification Driven Development (SDD)

The specification as an executable contract, to master the production quality of agents.

Mapping processes is not enough if every generation remains a non-reproducible act: **to tame agentic AI at scale requires a discipline that makes quality predictable**. Enter SDD.

A lineage: BDD, ATDD

SDD extends the legacy of **BDD** (*Behaviour-Driven Development*, Dan North, 2003) and **ATDD** (*Acceptance Test-Driven Development*): the executable specification was already the contract shared between business and technical sides. SDD's contribution in the agentic era: it becomes the **single source of generation** for code by agents, no longer just a validation test.

5.1. The failure of “Vibe Coding”

Dominant practices — *Vibe Coding* — rest on an intuitive interaction between developer and AI: informal prompt, generation, manual tweak. At prototype scale, that is enough; at industrial scale, every interaction is unique and non-reproducible — quality depends on individual expertise, standards are not shareable, verification rests on intuition. **Vibe coding leads to the maintenance wall.**

5.2. The rise of SDD



Thoughtworks Technology Radar

The Thoughtworks radar Vol. 34 (April 2026) confirms the trajectory: *Curated shared instructions for software teams* moves to **Adopt** — aligned with our principle of **curated shared instructions**. SDD, identified as early as Vol. 33, remains in *Assess* with growing maturity: the specification establishes itself as the **executable contract** that precedes and validates code generation.

SDD elevates the specification to the rank of **binding source of truth** — **code is no longer the primary artifact**. Birgitta Böckeler (*SDD: the 3 tools*, Martin Fowler, 2025) distinguishes three postures that mark adoption.



Spec-first

The specification is written first and used as a structured prompt to generate the code. But it is not maintained over time: it can silently drift away from code edited by hand thereafter.



Spec-anchored

Spec and code stay continuously synchronised. The code remains editable, but every change is anchored in an updated specification; divergences are detected.



Spec-as-source

Only the specification is edited. Code is a generated artifact, comparable to a compiler's binary; modifying the specification suffices to regenerate the system.

This adoption is what we aim for at Zenika. Targeting **spec-as-source** orients the organisation and methods towards **uncompromising quality**: the specification becomes the single source of audit, review, and reasoning. This climb in expectation is **gradual** depending on team maturity — start in *spec-first*, consolidate in *spec-anchored*, then aim for *spec-as-source*. But, as with open source, it remains **mandatory and possible to go into the code**, read it, and understand it: the readability of generated code is non-negotiable — security, auditability, compliance demand it. It is **craftsmanship** that guarantees the maintainability of code and architecture, where the specification guarantees that of the product. These two disciplines, inseparable, make the system maintainable over time, whatever its future maintainer — human or AI.

5.3. Tooling: from ideation to orchestration

The SDD ecosystem organises itself around a core suite of complementary tools. **GitHub Spec Kit** offers an open structure to formalise artifacts in the repository, with a clear *spec* → *plan* → *tasks* → *implementation* cycle extensible to brownfield projects. As further examples, **OpenSpec** brings a versioned spec *delta* logic, useful to handle each evolution as an explicit change; **BMAD** formalises agent personas (PM, Architect, Dev...) collaborating through a structured workflow.

Beyond the tool, SDD imposes a discipline: every change is formulated as a specification *delta* (intent, impacted requirements, scenarios to verify) before code generation — first what changes, then how it is implemented. This sequence makes cross-review between humans and agents practicable, and preserves the decision history of every change — work that our tool **Zenflow** carries further with its **dedicated agents geared towards spec-as-source**.



Zenika Zenflow

Zenflow (open-source) materialises our agentic practices as reusable starters: **multi-tool** compatibility (Copilot, OpenCode, Claude Code), **multi-stack** back/front (Java/Spring, Python/FastAPI, Go/Gin, React, Next.js), and a proven **orchestration workflow** (an *Orchestrator* agent that delegates to specialised sub-agents). Init in two commands, artifacts versioned in the repository — strictly aligned with the curated shared instructions principle.



Chapter 6 — The AI IDP (Internal Developer Platform)

The orchestration layer that decouples workflow from provider.

Once the method is set (SDD), it still needs to be toolled. The AI Internal Developer Platform (AI IDP) is the platform that industrialises this discipline and **shifts agentic AI from an absorbed cost to a piloted asset**. It sits between development tools (code agents, specialised assistants) and model providers (cloud APIs or local models), and orchestrates the entire AI usage cycle in the organisation.

An agentic product cycle, not just a developer cycle

The PDLC reaches beyond code. Upstream, AI accelerates **product discovery**, the writing of *user stories*, and *backlog* prioritisation; agile rituals become synchronisation points between humans and agents. Downstream, **augmented deployment** (agent-piloted feature flags, observed *canary release*, automated rollback) and **feedback collection** (product telemetry, user verbatim, NPS) close the loop back into the specification through agentic AI. The AI IDP tools the technical chain; it is the enterprise AI agents that orchestrate the whole — product, project, delivery, ops.

No single product today covers the full scope. The AI IDP is implemented by **assembling** four complementary bricks, each pluggable and replaceable independently of the others.

6.1. AI Gateway: routing and control

The AI Gateway centralises LLM traffic. All calls from agents, IDEs, and internal services to models transit through this brick, which makes it possible to handle in one place authentication, multi-provider routing, semantic caching, rate limiting, observability (latency, cost, success), and compliance (sensitive data filtering).

On the implementation side, **Agentgateway** (oriented towards agent-to-agent and MCP traffic), **Kong AI Gateway** (extension of the Kong gateway with LLM plugins), and **LiteLLM** (lightweight proxy compatible with OpenAI/Anthropic/Bedrock/local models) are open references. The decoupling achieved here is fundamental: changing model provider has zero impact on downstream tools, since they all address the gateway and not provider APIs. In a Singapore context, this gateway layer becomes the practical mechanism to route workloads to US-hosted, China-hosted, or locally-hosted models based on sensitivity, latency, cost, and jurisdictional profile — without rewriting downstream code.

6.2. Agent Registry and MCP

The Agent Registry catalogues the capabilities of agents available in the organisation: their roles, the models they require, the permissions they hold, the history of their usage. It plays for agents the role a *service registry* plays for microservices.

To this catalogue adds the standardisation of access to data and tools via the **Model Context Protocol (MCP)**. MCP normalises how an agent exposes and consumes context sources: code repository, knowledge base, ticketing, telemetry, analysis tools. An MCP integration is reusable from one agent to another, from one tool to another.



This registry and MCP combination turns a scattered agent ecosystem into a coherent platform. It conditions the ability to quickly deploy new use cases without endlessly reinventing integrations.

6.3. Context Factory

The Context Factory is the **RAG engine** of the platform: it feeds agents with the exact business context they need to produce relevant work.

Concretely, it indexes internal sources (code, documentation, tickets, runbooks, data schemas) in a vector database — **Qdrant**, **Weaviate**, or equivalent — and exposes access points letting an agent semantically query that corpus in real time. *Skills* and *rules* specific to a domain (security, accessibility, FinOps, performance) are versioned and composed on the fly according to the task's context.

The stake is twofold. First, **reduce hallucinations** by anchoring every generation on facts from the real system rather than the model's priors. Second, **diffuse the standards**: an agent querying the Context Factory automatically benefits from the company's shared rules, without having to reinject them in every prompt. The Thoughtworks radar Vol. 34 (April 2026) accordingly elevated **context engineering** to the *Adopt* tier — the context window is now a *design surface* in its own right, no longer a mere prompt variable.

6.4. Prompt Versioning & FinOps

Prompts are not throwaway character strings — they are **engineering artifacts** that deserve the same care as application code.



Versioning

Each system prompt, each agent template, is versioned, tested on canonical case sets, and promoted in a controlled fashion from one environment to another. A quality regression must be detected before production, not discovered by users.



FinOps

Token consumption is monitored per agent, per team, per use case. Alerts and quotas avoid budget overruns linked to an agent in a loop, an over-long context, or unexpected usage.



Guardrails

Security policies — sensitive data filtering, blocking of injected prompts, limitation of risky actions — are applied at the input and output of model calls. They protect both the organisation (intellectual property leakage) and the consuming applications (injection, exfiltration).

This fourth brick closes the AI IDP loop: without versioned prompt management, cost control, and guardrails, the platform remains exposed to drift. And it is this pluggable assembly, where each brick evolves independently, that produces the switchability promised in Chapter 2.



Chapter 7 — PDLC governance and metrics

Securing the ROI of a complex ecosystem: feedback loops, compliance, and data-driven steering.

AI-assisted software engineering demands a nervous system: measurable **feedback loops** that turn intuition into steering, and **compliance guardrails** that make technical choices defensible against regulation. PDLC governance is not a bureaucratic layer pasted on top of delivery — it is the condition for the individual gains measured in Chapter 1 to become durable organisational performance.

7.1. Measuring delivery: DORA and its extensions

DORA metrics (Deployment Frequency, Lead Time for Changes, Change Failure Rate, Time to Restore Service) remain the reference for measuring delivery performance. The programme is enriched in 2025 with a dedicated **AI Capabilities Model**, which adds capabilities specific to AI-assisted development; then, in 2026, with the modelling of the adoption trajectory as a **J-Curve**: a temporary trough — the “tuition cost” of the transformation, due to the *learning curve*, the *verification tax*, and pipeline adaptation — precedes exponential growth.

We observe this J-Curve on the ground. With our customers, the adoption of agentic AI is systematically accompanied by a 6- to 12-month dip whose components precisely overlap the segments quantified in Chapter 2 (figure 2): methodology and tuning risk, infrastructure to put in place, MLOps skill ramp-up. Continuous DORA tracking turns this expected dip into a **piloted investment** rather than a budget surprise — and lets organisations avoid the trap of stopping the effort during the learning phase by misreading it as a failure.

Beyond canonical metrics, steering of the Zenika framework relies on proprietary indicators that reveal the real value of AI — where DORA alone may mask drift.

Zenika's key AI indicators

Verification rate	Percentage of AI-generated code validated by deterministic tests before merge.
Specifications/code ratio	Share of SDD artifacts relative to directly generated code.
Switchability score	Measured ability to switch between AI providers with no delivery impact.
Diffusion index	Adoption of curated shared instructions and Skills across the organisation.



7.2. Security and compliance: OWASP AIMA

PDLC security with AI assistance introduces new attack surfaces: *prompt injection*, data exfiltration through tools, supply chain compromise on models, *data poisoning* of context. These risks require an adapted reference.

The **OWASP AIMA** framework (AI Maturity Assessment) offers a structured grid to evaluate an organisation's AI security maturity. **AI Verify** (IMDA + AI Verify Foundation) provides the matching enterprise testing toolkit, with technical tests (robustness, bias, explainability) and process checks designed to satisfy local IMDA expectations while remaining crosswalk-compatible with US-side patterns (NIST AI RMF). Combined with classical controls (static analysis, dependency scanners, agentic review), this pairing lets AI security sit in a continuous improvement loop rather than a one-shot checklist.



The strict verification rule

No AI-generated code reaches production without crossing the **deterministic verification**: static analysis, unit and property tests, linters, agentic review distinct from the generating agent. This is the concrete answer to the finding that only 48% of developers systematically verify code before they commit (Sonar 2026).

7.3. The “AI-augmented CI/CD” pipeline

Integration of this governance into existing pipelines (GitHub Actions, GitLab CI) happens by adding steps — specification agent, generation under context, deterministic *quality gates*, agentic review distinct from the generating agent, final human review — without replacing the existing setup. Each step is a *quality gate*: if a threshold is not met, the pipeline stops and the agent is relaunched with feedback from the failing step. This automatic loop reduces correction time and concretely materialises what PDLC governance means in the AI era.

Conclusion — An AI maturity checklist for CTOs

Agentic AI-assisted development is no longer optional: it is an inevitable transformation. The question is no longer whether your organisation will adopt AI in its development cycle, but how it will do so — opportunistically and dependently, or methodically and independently.

8.1. The 7 questions to ask yourself

To assess your level of readiness, answer these seven questions.

1. Do your teams use a structured methodological framework (such as SDD) or an intuitive, non-reproducible approach?
2. Is your tooling decoupled from the model provider via an **AI IDP** (gateway, registry, context factory, prompt versioning)?
3. Can you switch to local Open Weight models in less than 48 hours with no impact on delivery?
4. Are your requirements codified in **curated shared instructions** versioned alongside the code?
5. Does every line of AI-generated code go through a **deterministic verification** before merge (tests, static analysis, agentic review)?
6. Do your *brown field* projects have a structured context (Knowledge Graph, MCP) that can be accessed during code generation?
7. Are you continuously measuring augmented DORA metrics (verification rate, specifications/code ratio, switchability score)?



Reading the score

0–2 “yes” — Strategic urgency. Risk of joining the 40% of agentic AI projects that Gartner forecasts will be abandoned by the end of 2027.

3–5 “yes” — On track, structure it. Foundations exist; what is missing is overall coherence.

6–7 “yes” — Excellent, industrialise it. You are in a position to become a reference in your ecosystem.

8.2. Taking action

The methodology and the framework exist

Only the decision to adopt them is missing. Zenika supports you at every step of this transformation.

Contact us for an AI maturity diagnostic of your organisation.



Appendix

9.1. Sources and bibliography

9.1.1 Reports and studies

- DORA — *The ROI of AI-Assisted Software Development 2026*, Google Cloud. <https://cloud.google.com/resources/content/dora-roi-of-ai-assisted-software-development>
- DORA — *State of AI Assisted Software Development 2025*, Google Cloud. <https://cloud.google.com/resources/content/2025-dora-ai-assisted-software-development-report>
- SonarSource — *State of Code Developer Survey 2026*. <https://www.sonarsource.com/resources/developer-survey-report/>
- Stack Overflow — *Developer Survey 2025*. <https://survey.stackoverflow.co/2025/ai/>
- McKinsey — *The State of AI in 2025: Agents, Innovation, and Transformation*. <https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai>
- McKinsey — *Unlocking the Value of AI in Software Development*. <https://www.mckinsey.com/industries/technology-media-and-telecommunications/our-insights/unlocking-the-value-of-ai-in-software-development>
- Gartner — *40% of Enterprise Applications Will Feature Task-Specific AI Agents by 2026*. <https://www.gartner.com/en/newsroom/press-releases/2025-08-26-gartner-predicts-40-percent-of-enterprise-apps-will-feature-task-specific-ai-agents-by-2026-up-from-less-than-5-percent-in-2025>
- Thoughtworks — *Technology Radar Vol. 34*, April 2026. <https://www.thoughtworks.com/radar/>
- MIT Technology Review — *Everyone Wants AI Sovereignty. No One Can Truly Have It*. <https://www.technologyreview.com/2026/01/21/1131513/everyone-wants-ai-sovereignty-no-one-can-truly-have-it/>
- Birgitta Böckeler — *SDD: the 3 tools* (Martin Fowler blog). <https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html>

9.1.2 Regulatory and governance references — Singapore

- IMDA — *Model AI Governance Framework for Agentic AI*, Jan. 2026. <https://www.imda.gov.sg/-/media/imda/files/about/emerging-tech-and-research/artificial-intelligence/mgf-for-agentic-ai.pdf>
- IMDA — *Green Data Centre Roadmap*, May 2024. <https://www.imda.gov.sg/how-we-can-help/green-dc-roadmap>
- PDPC — *Singapore's Approach to AI Governance*. <https://www.pdpc.gov.sg/help-and-resources/2020/01/model-ai-governance-framework>
- AI Verify Foundation — *AI Verify Testing Framework*. <https://aiverifyfoundation.sg/>
- AI Singapore — *National AI Programme*. <https://aisingapore.org/>
- GovTech Singapore — *Government on Commercial Cloud (GCC)*. <https://www.tech.gov.sg/products-and-services/for-government-agencies/software-development/government-on-commercial-cloud/>
- MAS — *Notice FSM-N06 on Cyber Hygiene* (replaces the cancelled Notice 655, effective 10 May 2024). <https://www.mas.gov.sg/regulation/notices/notice-fsm-n06>
- MAS — *Guidelines on Technology Risk Management (TRM) and Guidelines on Outsourcing* (data-residency and cross-border data obligations for financial institutions). <https://www.mas.gov.sg/regulation/guidelines/technology-risk-management-guidelines>



- MOH — *HealthTech Instruction Manual* (sectoral guidance on health data handling). <https://hpa.gov.sg/healthtech-instruction-manual/>
- Cyber Security Agency of Singapore — *Cybersecurity Act 2018* (Critical Information Infrastructure designation). <https://www.csa.gov.sg/legislation/Cybersecurity-Act>
- EMA Singapore — *Electricity tariffs and contestable retail market*. <https://www.ema.gov.sg/consumer-information/electricity/buying-electricity>

9.1.3 Regulatory and governance references — US and China

- US Bureau of Industry and Security — *Export Administration Regulations (EAR), Advanced Computing and Semiconductor Manufacturing Controls*. <https://www.bis.doc.gov/index.php/policy-guidance/advanced-computing>
- US Department of Justice — *Clarifying Lawful Overseas Use of Data Act (CLOUD Act)*, 2018. <https://www.justice.gov/criminal-oia/cloud-act-resources>
- NIST — *AI Risk Management Framework (AI RMF 1.0)*. <https://www.nist.gov/itl/ai-risk-management-framework>
- Cyberspace Administration of China — *Measures for the Security Assessment of Outbound Data Transfers*, 2023.

9.1.4 Singapore data-centre market and infrastructure sources

- CBRE — *Global Data Center Trends 2025*. <https://www.cbre.com/insights/reports/global-data-center-trends-2025>
- Cushman & Wakefield - *Global Data Center Market Comparison 2024*. <https://www.cushmanwakefield.com/en/insights/global-data-center-market-comparison>

9.1.5 Tools, frameworks, and standards

- GitHub Spec Kit — <https://github.com/github/spec-kit>
- OpenSpec — <https://openspec.dev/>
- BMAD Method — <https://docs.bmad-method.org/>
- Zenika Zenflow — <https://github.com/zenika-open-source>
- Agentgateway — <https://agentgateway.dev/>
- Kong AI Gateway — <https://konghq.com/products/kong-ai-gateway>
- LiteLLM — <https://github.com/BerriAI/litellm>
- Qdrant — <https://qdrant.tech/>
- Weaviate — <https://weaviate.io/>
- OWASP AI Maturity Assessment (AIMA) — <https://owasp.org/www-project-ai-maturity-assessment/>

Authors



Olivier Coupelon

Head of AI Zenika FR



Michael Isvy

Head of Software
Engineering SG



Matthias Dugué

Head of Software
Engineering FR



Yu Jia Cheong

Senior Software
Consultant SG



Adrien Nortain

CTO & Partner FR



**Norbert Jeff
Nadir**

AI Ops & Change
Management FR

info@zenika.com

